The team tried to read a music sheet

What we asked for, who took responsibility, and why the work started with chord recognition.

We asked the team to turn images of music into editable kern files for a church band. The files needed to preserve the melody and chord symbols, and support correct transposition. The team was also expected to investigate failures, measure its progress, and tell us honestly if the result was not good enough.

For this first session, the research lead had enough budget to commission four experiments. The session was configured to use seven excerpts, one from each song group, as a small working set. A full acceptance review would require all 32 excerpts, no measured melody or chord errors, valid files, verified transposition, audited chord timing, and a musician's review. This small trial could inform the next decision, but could not pass that full acceptance test.

The research lead chose a narrow first job

The lead first checked the images, existing benchmark results, and a local format audit. The enlarged-image Audiveris method was the strongest melody starting point among the compared methods on this trial: 87 melody errors across 130 reference events. However, it matched only 8 of 38 chord labels. Another existing approach, greedy Transcoda, had produced excessive extra notes.

That suggested a limited experiment: keep the existing melody and improve how printed chord text becomes structured musical information. The lead's idea was that some missing chords might already be readable as text, without replacing the whole recognition method.

Who did what

The research lead chose the next job and reviewed the evidence. **The chord engineer** tackled printed chord text. **The integration engineer** tried to produce complete melody-and-chord files. **The runtime engineer** investigated why recognition would not start. **The platform engineer** attempted the final targeted repair.

How to read this story: these job titles are editorial names for the recorded software roles, not real employees or newly added specialists. They worked in sequence. Engineers returned work to the lead; they did not directly manage one another. The review desk was automated scoring code, not another person. Requests below are paraphrased from the saved assignments, not invented dialogue.

The chord engineer found a missing handoff

First assignment: recover more chord labels while keeping the melody unchanged. About 8.6 active minutes.

The request arrived with an assumption

The lead asked the chord engineer to read chord symbols above the staff, convert them into an editable harmony part, and preserve the existing melody. Success meant seven complete files, more correct chords than the baseline, and no increase in melody errors. Uncertain chord timing had to be flagged rather than guessed.

The engineer received the assignment and began by looking for the score images and the existing melody outputs. The images were present; the melody files were not. The engineer explicitly recognized the problem: aggregate benchmark scores could not reconstruct the missing music. The intended experiment could no longer demonstrate that the melody had stayed unchanged.

The engineer continued with the part that was possible

The engineer pursued conservative optical character recognition: enlarge the images, inspect the region above each staff, and accept text only when it matched a strict chord grammar. That was an implementation of the lead's idea, not a new research direction invented independently. The engineer prepared a contact sheet, inspected the images, and built the chord-reading and conversion stage.

Much of the time went into setup. Dependency installation ran out of space, the text-recognition executable was unavailable, and library repairs were needed. The engineer eventually extracted public software packages into the writable workspace instead of relying on system-wide installation.

The submitted work was useful, but incomplete

The engineer produced seven incomplete outputs containing 15 accepted chord candidates. Local checks on synthetic examples passed, but those checks did not establish that the actual complete transcriptions were usable. The first submission was rejected because the required source files were missing; the engineer corrected the submission on the final attempt.

The automated review found 13 correctly matched chords, up from 8. Chord errors fell from 30 to 25. But there were no melody events, all 130 reference melody events counted as errors, and none of the seven complete files passed parsing. The engineer's own final report acknowledged the missing melody and did not claim a successful transcription.

The handoff: the chord engineer reported back to the lead. After seeing the measurements, the lead chose an integration engineer to recover real melody and build complete files. The lead kept the promising chord work, but stopped treating it as a successful end-to-end result.

The integration engineer tried to restore the melody

Second assignment: run melody recognition afresh and combine it with the chord work. About 8.8 active minutes.

The lead changed the request because of the review

The lead saw that chord recognition had improved, but missing melody had made the output unusable. Further chord tuning would not fix that. The new request was to keep the chord method unchanged, run Audiveris on enlarged images, convert actual recognized notes, and independently check the complete files. The engineer was explicitly told not to assume that baseline outputs existed or to substitute dummy notes.

Before dispatching this job, the lead tried to classify it as a debugging task. The system rejected that request because the earlier work had completed and been evaluated, rather than being classified as an execution failure. The lead then submitted it as an improvement job. This was an administrative correction, not an additional experiment.

The engineer found why the earlier checks were too weak

The integration engineer inspected the inherited code. When melody was missing, it had emitted an empty melody part; its local checks could accept a parsed score with no musical events. The engineer concluded that the next test needed both real recognition output and a nonempty, independently readable transcription. The exact protected-reader failure was still not confirmed.

The engineer tried to install and run a fresh recognition pipeline. Setup again became the main work: temporary storage filled up, one Java download was denied, and the first Java version was incompatible. The engineer moved temporary installation work into writable storage, obtained another runtime, replaced Java 21 with Java 25, and investigated native-library startup failures.

Recognition never produced the music to integrate

Despite those changes, Audiveris produced no genuine MusicXML exports for the seven images. Without recognized notes, the engineer could not validate complete transcriptions or test transposition on real outputs. The submitted files were explicitly marked failure placeholders, not invented melody.

The automated review again found no parseable complete files. Melody errors stayed at 130 out of 130, and chord errors rose to 38 out of 38. The earlier chord gain had not survived in the submitted results. Writing integration code and successfully launching shell commands had not produced a working transcription pipeline.

The handoff: the engineer returned the failed integration and its diagnostics to the lead. The lead decided that recognition startup was the immediate bottleneck, and assigned a runtime engineer to obtain one real export before attempting the whole set again.

The runtime engineer narrowed the failure

Third assignment: diagnose startup, then prove recognition works on one image. About 5.0 active minutes.

The lead asked for a smaller proof

Two experiments remained. The lead could have changed recognition methods, but chose to investigate the execution problem first. Its recorded reason was that recognition had never started, so the poor transcription scores did not yet show what a functioning version of the method could do.

The runtime engineer received a narrower request: inspect the inherited startup diagnostics, correct only the implicated runtime or library problem, and obtain a real MusicXML export from one image within a 180-second preflight. Only after that succeeded should the engineer process all seven images. If it failed, the engineer was to stop and preserve the exact error, without generating placeholder transcriptions.

The handoff contained code, but not a ready environment

The engineer searched the inherited launch instructions, diagnostics and runtime paths. The earlier environment was not fully available. Source and selected configuration had been passed on, but installed runtimes and all supporting diagnostic files had not. The engineer had to recover the launch details and restore Java and Audiveris before reproducing the problem.

The engineer also had to request information that had been omitted from a shortened log response. This mattered because the assignment's preflight deadline was running while setup and inspection continued. The team had compact logs, but sometimes needed extra requests to recover the specific fact required for the next decision.

The engineer identified a concrete blocker

After restoring Java 25 and Audiveris 5.11, the engineer captured a native-library error: the application could not load GTK 3. No MusicXML export was produced. The engineer stopped, preserved the startup exception and runtime details, and moved inherited placeholders aside so they would not be mistaken for newly recognized music.

This was diagnostic progress, not a transcription improvement. The job produced no predictions, so there was no new accuracy score. The engineer reported that complete-file parsing, transposition on recognized notes, and chord timing remained unverified.

The handoff: the runtime engineer reported the missing GTK library to the lead. With one experiment left, the lead chose a platform engineer for a targeted repair. It judged the specific missing library more actionable than another broad setup attempt. The engineer did not choose the successor or issue that assignment.

The platform engineer fixed one dependency

Fourth assignment: make GTK load, then obtain a genuine recognition export. About 6.1 active minutes.

The last request was specific

The lead asked the platform engineer to reproduce the GTK failure, install or otherwise provide the missing library, verify that it loaded, and attempt an export within a 240-second preflight. The recognition method and chord settings were to stay unchanged. A full seven-image run would happen only if that first export succeeded.

The engineer received the earlier source and diagnostics, but again had to inspect restoration instructions and missing runtime paths. During this work, one model-service call failed before producing an action. The work continued afterward; that interruption was separate from the music software's failure.

Normal installation was blocked, so the engineer adapted

System package installation failed because the workspace did not have permission to lock the package manager's directories. The engineer therefore downloaded distribution packages and extracted their contents locally. An initial attempt failed on a missing user-account lookup; the engineer removed that dependency and retried.

This time, a fresh Python process loaded the GTK library and queried its version. That established a real, limited result: the library was available through the local package extraction. It did not establish that Java and Audiveris could now use it successfully, or that recognition could export music.

The deadline expired before the decisive test completed

By the time the export wrapper reached its final attempt, only about 3.43 seconds remained under the original deadline. The wrapper timed out at approximately 240 seconds overall. No MusicXML export and no kern predictions were produced. The record did not establish a specific new native-library blocker after the GTK check.

The engineer stopped launching recognition and inspected the saved verification and timeout evidence. The final report distinguished the successful library-loading check from the unproven recognition pipeline. It did not claim a musical improvement, and there was no new accuracy measurement to support one.

The handoff: the platform engineer returned the diagnostics to the lead. There was no next engineer: all four experiments had been used. The lead ended the session, kept the first engineer's partial chord work as the best measured candidate, and recorded that the overall goal remained unmet.

What the team actually learned

The team followed the evidence, but spent its remaining experiments making the same method run.

The session lasted about 32.6 active minutes and used 68 model calls. It ended because the four-experiment allowance had been consumed, not because all time or model-call budget had run out. The lead retained the first chord-reading result as partial evidence, rather than promoting it as a finished product.

Work reviewed	Melody errors	Chord errors	Complete files
Existing enlarged baseline	87 / 130	30 / 38	Not compared here
Chord engineer	130 / 130	25 / 38	0 / 7 parseable
Integration engineer	130 / 130	38 / 38	0 / 7 parseable
Runtime / platform engineers	Not scored	Not scored	No predictions

Lower error counts are better. These are edit errors against the same seven development excerpts, not a held-out accuracy result. The first experiment improved chord labels, but worsened melody and did not produce usable complete files. Missing predictions in later work are failures, not zero-error results.

Why the lead’s decisions mattered

The lead did use results to change the work: first recover chords, then restore melody, then prove startup on one image, then repair a specific library. Each new request addressed an observed problem. But the lead never commissioned a fundamentally different recognition method. The remaining budget went into increasingly narrow repairs to the same approach.

What we should change before asking the team to try again

Make the first assignment executable. The chord engineer was asked to preserve melody that had not been supplied. The lead should verify that required inputs actually exist before handing over a job.

Hand over a reproducible working environment. Successive engineers repeatedly restored dependencies. Passing code without the environment consumed time that could have tested the research idea.

Prove a complete output early, and reserve room for another idea. A real, nonempty transcription should be the first milestone. Repeated setup failures should trigger a deliberate choice between more repair and a different recognition method. These are recommendations from this review, not changes already implemented.

Our conclusion: the team showed evidence-informed problem solving and honest reporting. It did not yet demonstrate a better end-to-end transcription system. The automated review prevented partial progress and successful setup commands from being mistaken for success.

Source basis: the preserved 28 October 2026 execution record, original assignments, coordinator decisions, worker histories, protected measurements and session assessment. This narrative reorganizes those records into team language. It introduces no new run, dialogue, staff members or implementation changes. The original detailed report and saved implementation remain unchanged.